

LAPORAN PRAKTIKUM W3

Class, Object dan Encapsulation Part 2

Diajukan untuk memenuhi salah satu tugas praktikum mata kuliah Pemrograman Berorientasi Objek.



Disusun Oleh:
Rainissa Azizah (251511059)
Jurusan Teknik Komputer dan Informatika

Program Studi D-3 Teknik Informatika
Politeknik Negeri Bandung
2026

A. Deskripsi Aplikasi

Aplikasi sistem informasi konsultasi rumah sakit sederhana ini dijalankan melalui console dan dikembangkan dengan Java. Interaksi anatar dokter dan pasien dapat diatur secara digital dengan mencakup tahapan mendaftar melakukan pemeriksaan medis, sampai menentukan diagnosis akhir. Aplikasi ini juga menerapkan enkapsulasi, yaitu:

1. Perlindungan Atribut

Seluruh variabel local di class Pasien (nama, no. rekam medis, umur, dan keluhan) dan class Dokter (nama dokter dan spesialisasi) dikunci menggunakan access modifier private supaya tidak bisa diakses sembarangan.

2. Getter dan Setter

Nilai variabel dibaca dari luar class dapat menggunakan method getter. Sedangkan, pengubahan nilai dilakukan lewat method setter yang dilengkapi pemeriksaan khusus. Misalnya pada variabel umur, nilai hanya diterima jika berada di rentang 1-129 tahun untuk mencegah input tidak masuk akal.

3. Keamanan Identitas

Variabel nama dan noRekamMedis diberi label final agar nilainya terkunci secara permanen begitu objek pasien baru dibuat.

4. Interaksi Objek

Hubungan antara Pasien dan Dokter dijalankan melalui fungsi konsultasi(Dokter dokter) yang memanfaatkan perintah this untuk meneruskan data pasien ke dokter.

B. Struktur Package

Class-class yang ada pada aplikasi ini dikelompokkan ke dalam tiga *package* sesuai dengan tanggung jawabnya masing-masing, yaitu:

1. `id.ac.polban.model` berisi *class* Pasien dan Dokter yang murni menyimpan data, seperti atribut, getter, dan setter).
2. `id.ac.polban.service` berisi *class* RumahSakitService yang menangani logika proses, seperti pendaftaran pasien/dokter dan proses konsultasi.
3. `app` berisi *class* Main sebagai *entry point* program.

Pemisahan ini membuat struktur kode menjadi lebih rapi dan jelas pembagian tanggung jawabnya.

C. Relasi Antar Class

1. Dependency ("uses-a")

Penerapan *Dependency* terlihat saat *class* RumahSakitService menjalankan *method* konsultasi(Pasien pasien, Dokter dokter) dan saat *class* Dokter menjalankan *method* periksa(Pasien pasien). Objek luar yang masuk lewat parameter hanya dibutuhkan selama *method* bekerja. Begitu eksekusi kode selesai, keterkaitan antar-objek langsung hilang karena tidak disimpan di dalam variabel *class*.

2. Aggregation ("has-a")

Untuk relasi *aggregation*, keterkaitannya ditunjukkan oleh atribut daftarPasien dan daftarDokter berformat List<Pasien> dan List<Dokter> pada *class* RumahSakitService. Saat fungsi daftarkanPasien() dan daftarkanDokter() dipanggil, objek akan dicatat ke dalam daftar secara permanen. Walaupun menjadi bagian dari data RumahSakitService, objek Pasien dan Dokter punya alur hidup terpisah sehingga tidak bergantung sepenuhnya pada *class* pengelolanya.

D. Packaging ke dalam JAR File

Aplikasi ini dikemas ke dalam bentuk file JAR (RumahSakitApp.jar) menggunakan javac dan jar dari JDK, dengan langkah-langkah sebagai berikut:

1. Compile seluruh source code ke folder bin:

```
javac -d bin src/id/ac/polban/model/*.java  
src/id/ac/polban/service/*.java src/app/Main.java
```

2. Membuat file manifest.txt yang berisi informasi class utama program:

```
Main-Class: app.Main
```

3. Membuat file JAR menggunakan manifest tersebut:

```
jar cfm RumahSakitApp.jar src/manifest.txt -C bin .
```

4. Menjalankan file JAR yang telah dibuat:

```
java -jar RumahSakitApp.jar
```

Untuk memastikan JAR tersebut dapat digunakan secara mandiri tanpa bergantung pada source code atau IDE, dilakukan pengujian dengan membuat sebuah project baru yang hanya memuat file RumahSakitApp.jar (tanpa file .java sama sekali), lalu menjalankannya melalui perintah `java -jar` dari project tersebut. Hasilnya, aplikasi tetap dapat berjalan dengan baik, membuktikan bahwa file JAR bersifat portable dan dapat dipindahkan atau digunakan di tempat lain.

```

PS C:\Users\User\OneDrive - Politeknik Negeri Bandung\Kuliah\Semester
3\PBO\PBO Praktikum\Praktikum PBO W3\TesJarRumahSakit> java -jar lib/
RumahSakitApp.jar
=====
SISTEM INFORMASI RUMAH SAKIT
=====
dr. Tirta berhasil didaftarkan sebagai dokter.
Rainissa Azizah berhasil didaftarkan sebagai pasien.

--- Data Awal Pasien ---
Nama      : Rainissa Azizah
No. Rekam Medis: RM-001
Umur      : 20 tahun
Keluhan   : demam dan pusing

--- Uji Coba Validasi Setter Umur ---
Umur tidak valid, gagal diubah.
Umur sesudah : 21 tahun

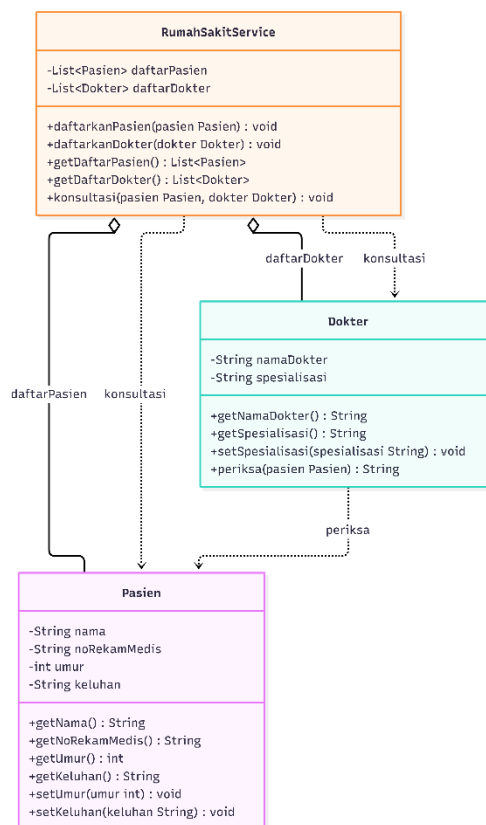
--- Proses Konsultasi Pasien ---
Rainissa Azizah mendaftar untuk diperiksa oleh dr. Tirta
Hasil diagnosis: Pasien Rainissa Azizah (demam dan pusing)
-> Diberi resep obat & istirahat oleh Dokter Umum

=====
SELESAI
=====

```

E. Class Diagram

Berikut adalah *class* diagram yang menggambarkan struktur, atribut, *method*, serta relasi antar *class* pada aplikasi ini. Diagram dibuat menggunakan Mermaid, dengan tiga *class* utama, yaitu Pasien, Dokter, dan RumahSakitService.



Pada diagram tersebut, garis dengan simbol wajik kosong menunjukkan relasi *Aggregation*, yaitu dari RumahSakitService ke Pasien (melalui atribut daftarPasien) dan ke Dokter (melalui atribut daftarDokter). Sementara itu, garis putus-putus dengan panah menunjukkan relasi *Dependency*, yaitu dari RumahSakitService ke Pasien dan Dokter (melalui method konsultasi()), serta dari Dokter ke Pasien (melalui method periksa()).

F. Source Code

1. Pasien.java

```
2. package id.ac.polban.model;
3.
4. public class Pasien {
5.     private final String nama;
6.     private final String noRekamMedis;
7.     private int umur;
8.     private String keluhan;
9.
10.    public Pasien(String nama, String noRekamMedis, int umur,
11.        String keluhan) {
12.        this.nama = nama;
13.        this.noRekamMedis = noRekamMedis;
14.        this.umur = umur;
15.        this.keluhan = keluhan;
16.    }
17.
18.    public String getNama() {
19.        return nama;
20.    }
21.    public String getNoRekamMedis() {
22.        return noRekamMedis;
23.    }
24.    public int getUmur() {
25.        return umur;
26.    }
27.    public String getKeluhan() {
28.        return keluhan;
29.    }
30.    public void setUmur(int umur) {
31.        if (umur > 0 && umur < 130) {
32.            this.umur = umur;
33.        } else {
34.            System.out.println("Umur tidak valid, gagal diubah.");
35.        }
36.    }
```

```

37.     public void setKeluhan(String keluhan) {
38.         this.keluhan = keluhan;
39.     }
40. }

```

2. Dokter.java

```

3. package id.ac.polban.model;
4.
5. public class Dokter {
6.     private final String namaDokter;
7.     private String spesialisasi;
8.
9.     public Dokter(String namaDokter, String spesialisasi) {
10.        this.namaDokter = namaDokter;
11.        this.spesialisasi = spesialisasi;
12.    }
13.
14.    public String getNamaDokter() {
15.        return namaDokter;
16.    }
17.    public String getSpesialisasi() {
18.        return spesialisasi;
19.    }
20.    public void setSpesialisasi(String spesialisasi) {
21.        this.spesialisasi = spesialisasi;
22.    }
23.    public String periksa(Pasien pasien) {
24.        return "Pasien " + pasien.getNama() + " (" + pasien.getKeluhan() +
25.            ")\n" +
26.            "-> Diberi resep obat & istirahat oleh " + spesialisasi;
27.    }

```

3. RumahSakitService.java

```

4. package id.ac.polban.service;
5.
6. import id.ac.polban.model.Dokter;
7. import id.ac.polban.model.Pasien;
8. import java.util.ArrayList;
9. import java.util.List;
10.
11. public class RumahSakitService {
12.     private final List<Pasien> daftarPasien;

```

```

13.     private final List<Dokter> daftarDokter;
14.
15.     public RumahSakitService(){
16.         this.daftarPasien = new ArrayList<>();
17.         this.daftarDokter = new ArrayList<>();
18.     }
19.     public void daftarkanPasien(Pasien pasien){
20.         daftarPasien.add(pasien);
21.         System.out.println(pasien.getNama()+" berhasil didaftarkan
    sebagai pasien.");
22.     }
23.     public void daftarkanDokter(Dokter dokter){
24.         daftarDokter.add(dokter);
25.         System.out.println(dokter.getNamaDokter()+" berhasil didaftarkan
    sebagai dokter.");
26.     }
27.     public List<Pasien> getDaftarPasien(){
28.         return daftarPasien;
29.     }
30.     public List<Dokter> getDaftarDokter(){
31.         return daftarDokter;
32.     }
33.     public void konsultasi(Pasien pasien, Dokter dokter) {
34.         System.out.println(pasien.getNama() + " mendaftar untuk
    diperiksa oleh " + dokter.getNamaDokter());
35.         String hasil = dokter.periksa(pasien);
36.         System.out.println("Hasil diagnosis: " + hasil);
37.     }
38. }

```

4. Main.java

```

5. package app;
6.
7. import id.ac.polban.model.Dokter;
8. import id.ac.polban.model.Pasien;
9. import id.ac.polban.service.RumahSakitService;
10.
11. public class Main {
12.     public static void main(String[] args) {
13.         System.out.println("=====");
14.         System.out.println("        SISTEM INFORMASI RUMAH SAKIT        ");
15.         System.out.println("=====");
16.
17.         Dokter dokter1 = new Dokter("dr. Tirta",

```

```

18.         "Dokter Umum");
19.         Pasien pasien1 = new Pasien("Rainissa Azizah",
20.             "RM-001", 20,
21.             "demam dan pusing");
22.
23.         RumahSakitService service = new RumahSakitService();
24.         service.daftarkanDokter(dokter1);
25.         service.daftarkanPasien(pasien1);
26.
27.         System.out.println("\n--- Data Awal Pasien ---");
28.         System.out.println("Nama           : "
29.             + pasien1.getNama());
30.         System.out.println("No. Rekam Medis: "
31.             + pasien1.getNoRekamMedis());
32.         System.out.println("Umur           : "
33.             + pasien1.getUmur() + " tahun");
34.         System.out.println("Keluhan        : "
35.             + pasien1.getKeluhan());
36.
37.         System.out.println("\n--- Uji Coba Validasi Setter Umur ---");
38.         pasien1.setUmur(-5); // percobaan input tidak valid
39.         pasien1.setUmur(21); // percobaan input valid
40.         System.out.println("Umur sesudah : " + pasien1.getUmur()
41.             + " tahun");
42.
43.         System.out.println("\n--- Proses Konsultasi Pasien ---");
44.         service.konsultasi(pasien1, dokter1);
45.
46.         System.out.println("\n=====");
47.         System.out.println("                        SELESAI                        ");
48.         System.out.println("=====");
49.     }
50. }

```

G. Hasil Implementasi

```

PS C:\Users\User\OneDrive - Politeknik Negeri Bandung\Kuliah\Semester 3\PBO
\PBO Praktikum\Praktikum PBO W3> java -jar TesJarRumahSakit/lib/RumahSakitApp
.jar
=====
SISTEM INFORMASI RUMAH SAKIT
=====
dr. Tirta berhasil didaftarkan sebagai dokter.
Rainissa Azizah berhasil didaftarkan sebagai pasien.

--- Data Awal Pasien ---
Nama      : Rainissa Azizah
No. Rekam Medis: RM-001
Umur      : 28 tahun
Keluhan   : demam dan pusing

--- Uji Coba Validasi Setter Umur ---
Umur tidak valid, gagal diubah.
Umur sesudah : 21 tahun

--- Proses Konsultasi Pasien ---
Rainissa Azizah mendaftar untuk diperiksa oleh dr. Tirta
Hasil diagnosis: Pasien Rainissa Azizah (demam dan pusing)
-> Diberi resep obat & istirahat oleh Dokter Umum

=====
SELESAI
=====

```

H. Lesson Learned

Dari tugas ini, saya belajar bagaimana mengorganisasikan kode ke dalam *package* agar tanggung jawab tiap *class* lebih jelas. Pemahaman mengenai perbedaan relasi *Dependency* dan *Aggregation* juga saya dapatkan dari cara objek diakses atau disimpan. Saya juga belajar mengemas aplikasi menjadi file JAR agar dapat dijalankan secara mandiri. Selama proses pengerjaan, saya sempat mengalami beberapa kendala teknis, seperti perintah `javac` dan `jar` yang tidak dikenali oleh terminal karena belum terdaftar pada PATH sistem. Dari pengalaman ini, saya menyadari bahwa memahami cara kerja tools dan environment yang digunakan sama pentingnya dengan menulis kode yang benar.

Link Repo

<https://github.com/rainissa/PBO-Praktik.git>